

Introduction to computing

Scope:-

↳ jis jis jaga variable ka daba nazar a raha hota ha-

Where can you see the variables?

def f(x):

y = 1

x = x + y

print('x = ', x)

print('p = ', p)

p = 20

x = 3

y = 2

z = f(x)

print('z = ', z)

print('x = ', x)

print('y = ', y)

~~Global~~
Object
function

P = 20

Global
frame

if you use ^{our} global variable for function it can effect our abstraction that is a bad thing and called

→ recall abstraction concept

(if we give input and the function change your ^{global} input value it called leak abstraction-)

Reading assignment:-

read section "1.3.5 choosing names" and "1.3.6 Functions as abstractions on the composing program."

```
def pressure(v, t, n):
```

```
    """
```

→ multiline string in
your function

```
    ~~~~~
```

```
    """
```

so when you call
help(pressure) it
defines what your
function do-

```
    k = 1.383.e
```

```
    return n * k * t / v
```

Compound Statements:-

Compound statements take the form:-

↓
small things combined e.g. function
make compound:

<header>:

<statement>

< " " >

...

<separating header>:

<statement>

< " " >

...

...

Conditionals:-

↳ when it evaluates it gives us a boolean value 0 or 1.

Also called branching statements - These allow you to make decision.

```
def absolute_value(x):
```

```
    """ compute abs(x) """
```

```
    if x >= 0:
```

```
        return x
```

```
    else:
```

```
        return -x
```

→ { if not $x \leq 0$:

return x

elif $x == 0$:

return 0

else

return -x

x
q/t



$$4 - 5 = -1$$

$$4 - 5 = -1$$

$$20 - 5 = 15$$

$$15 < 0.1$$

$$128$$

$$x$$

$$4 - 5 = -1$$

$$+1 < 0.1$$

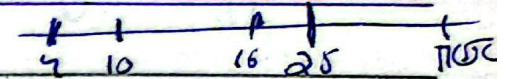
$$(10^2)$$

$$100 - 25 = 75$$

$$b - a - b$$

$$+ve \quad -ve$$

Back to square root problem



```
def sqrt(n, guess = 0):
```

```
    if good_guess(n):
        return guess
```

```
    else:
        indent it
        → new_guess = improve_guess()
        sqrt(n, new_guess)
```

```
def improve_guess(n, g):
```

```
    if g * g < n/2
```

```
    if
    return avg(n, g/n)
```

```
def good_enough(n, g):
```

```
    if (g * g - n) < 0.1:
        return true
```

```
    else:
        return false
```

$$25 - 16 = 9$$

$$9 < 0.1$$

$$0.1$$

$$25$$

imp (guess, n)

if guess < n

```
def avg(g, n):
```

```
    return (g + n) / 2
```

$$1.82 \sqrt{4}$$

$$1$$

$$g$$

Reading assignment:

Section 2.2 guttag's python book branching programs

chap 1.5 upto 1.5.4 conditional statement (meaning,
stop when you reach 15.5 (from composing program))

Square root algo:-

start guess

if guess good

return guess

if ^{not} improve guess

~~return~~

①

```
- def sqrt(x, guess) 1.8  
  -> if goodguess(x, guess) =  
      return guess
```

-> else

```
new_guess = improveguess(x, guess)
```

```
return sqrt(x, new_guess)
```

```
def goodguess(x, guess)  
  0.76
```

```
  if abs(guess * guess - x) < 0.1
```

```
    return true
```

```
  else
```

```
    return false
```

```
- def abs(x)
```

```
  if x < 0
```

```
    return -x
```

```
  else
```

```
    return x
```

```
def improveguess(x, guess)  
  1.8
```

```
  if guess < x
```

```
    return guess + 0.1
```

```
  else
```

```
    return guess - 0.1
```

```
-> v = sqrt(4, 1.8)
```

```
def sqrt(x, guess) 1.9
    if goodguess(x, guess) 1.9
        return guess
```

→ else

```
newguess = impguess(n, guess)
return sqrt(x, newguess)
```

```
def goodguess(n, guess) 0.39
    if abs(guess * guess - n) < 0.1
        return true
```

else

return false

```
def impguess(n, guess) 1.9
    if guess < n
        return guess * 1.1 - 2.0
    else
```

guess = 0.1

```
def sqrt(n, guess) 2.0
    if good(x, guess)
        return guess = 2.0
    else
```

```
newguess = impguess(n, guess)
return sqrt(n, newguess)
```

```
def impguess(n, guess) 2.0
    if (guess * guess - n) < 0.1
        return true
    else
```

return false

```
def impguess(n, guess)
    if guess < n
        return guess * 1.1
    else
        return guess - 0.1
```